



DEPARTMENT OF
COMPUTER SCIENCE



DECOMPOSITION AND MODULARIZATION

JEREMIAH O. BANDELE

PhD Electrical/Electronic Engineering
University of Nottingham



LEARNING OBJECTIVES

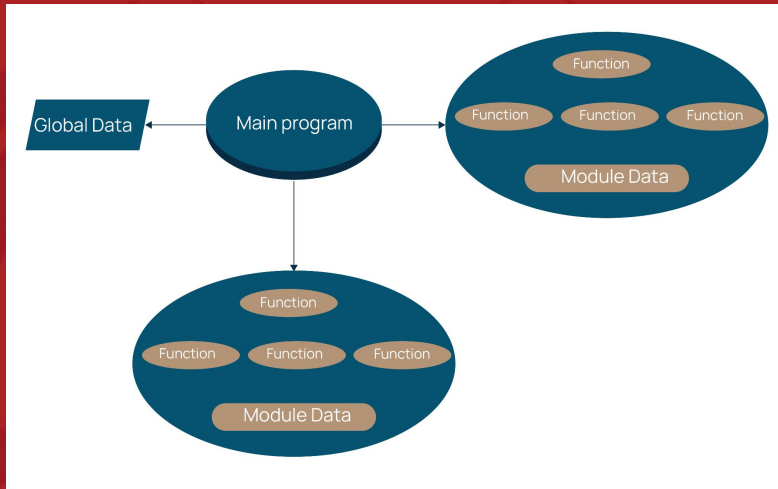
At the end of the lesson, you should be able to:

- Discuss the role of decomposition and modularisation in problem solving.
- Evaluate the impact of module independence on systems design.
- Discuss the structure and characteristics of modules.



MODULARISATION

Modularisation can be explained as the process of dividing a program's functionality into independent modules, such that each independent module has everything it needs to implement only a part of the program's functionality.





DECOMPOSITION

The meaning of decomposition is actually embedded in the definition of modularisation. By separating the codes in an application into different modules and functions, we are essentially decomposing the application into modules.



MODULARIZATION: WHY

Imagine that you are given a project to work on, and you have to write a function that would contain more than five thousand lines of code.

Can you even imagine how stressful it would be to change a part of the code?

Infact, changing a part of the code can even affect other parts of the code.

This is part of the reasons why we need modularization.



HOW MODULARISATION WORKS

With modularisation, we can separate the five thousand lines of code in our application into different modules and functions.

Infact, the modules and functions can be saved in separate files.



HOW MODULARISATION WORKS

Having this kind of configuration ensures that we are able to make changes to different parts of our code without affecting other parts of the code.

Also, since a software is most likely to have added functionality at some point, modularisation ensures that these additional functionalities can be added to the software in a way that quick and flexible manner.



MODULARISATION: ADVANTAGES

- It allows large programs to be written by several or different people.
- It is easier to manage and add smaller modules and functions to our program.
- Understanding a small module and function is easier.



MODULARISATION: ADVANTAGES

- We can easily use the modules and functions either in other parts of our program or in another program.
- Smaller modules and functions are easier and faster to design, develop, test and publish.



MODULARISATION: DISADVANTAGES

- Execution time may be longer
- Storage size may be increased
- Compilation and loading time may be longer
- Inter-module communication problems may be increased
- Demands more initial design time



MODULE INDEPENDENCE

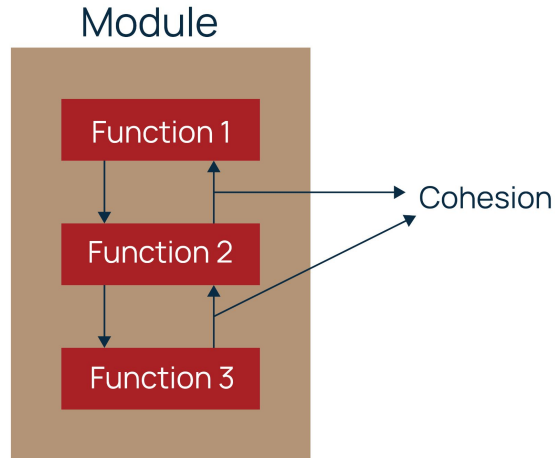
Just like functions, the single responsibility principle demands that modules should not have more than a single responsibility and that they should not depend on other modules.



MODULE INDEPENDENCE: COHESION

Cohesion describes the extent to which the functions in a module work together to achieve the ultimate aim of the module.

We usually want high cohesion between the functions in a module.

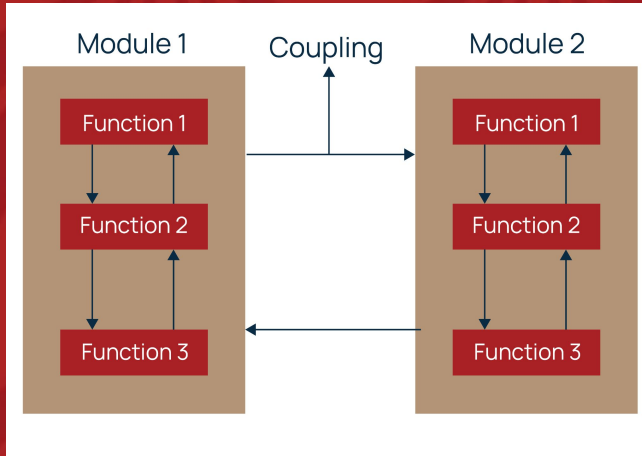




MODULE INDEPENDENCE: COUPLING

Coupling describes the extent to which modules depend on other modules before performing their function.

Since we want our modules to be independent, we usually want low coupling between the modules in a project.





MODULE INDEPENDENCE: CAR ANALOGY

A car transport passengers from one place to another.

What are the effects of cohesion and coupling on car parts such as steering wheel, radiator, battery, and brake?



INFORMATION HIDING

Modules should be specified and designed in such a way that the data structures and processing details of one module are not accessible to other modules.

They pass only that much information to each other, which is required to accomplish the software functions.

The way of hiding unnecessary details is referred to as information hiding.



INFORMATION HIDING: BENEFITS

- It emphasises communication through controlled interfaces.
- It decreases the probability of adverse effects.
- It restricts the effects of changes in one component on others.



REFACTORING

Refactoring is an important design activity that reduces the complexity of module design while keeping its behaviour or function unchanged.

Refactoring can be defined as a process of modifying a software system to improve the internal structure of design without changing its external behaviour.

During the refactoring process, the existing design is checked for any type of flaws like redundancy, poorly constructed algorithms and data structures, etc, in order to improve the design.



REFACTORING: EXAMPLE

For example, a design model might yield a component which exhibits low cohesion (like a component performs four functions that have a limited relationship with one another).

Software designers may decide to refactor the component into four different components, each exhibiting high cohesion.

This leads to easier integration, testing, and maintenance of the software components.



STRUCTURAL PARTITIONING

When the architectural style of a design follows a hierarchical nature, the structure of the program can be partitioned either horizontally or vertically.

Structural partitioning provides the following benefits.

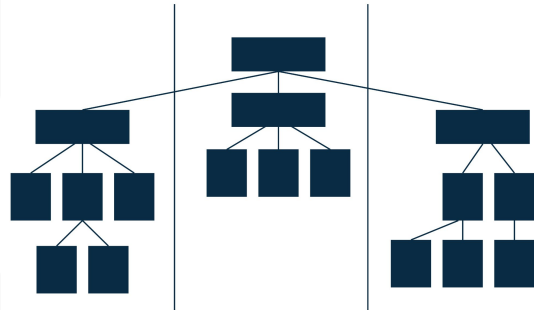
- The testing and maintenance of software becomes easier.
- The negative impacts spread slowly.
- The software can be extended easily.



STRUCTURAL PARTITIONING: HORIZONTAL

In horizontal partitioning, control modules are used to communicate between functions and execute the functions.

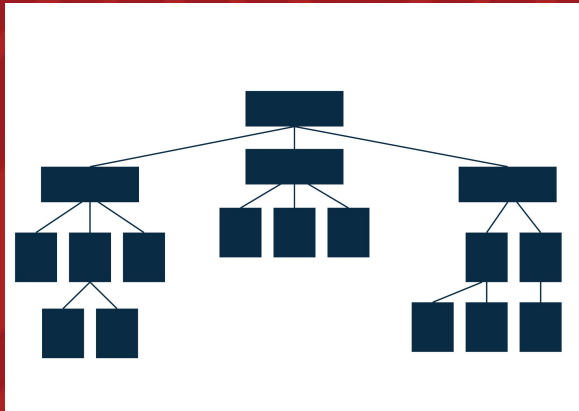
Horizontal partitioning requires the passing of more data across the module interface, which makes the control flow of the problem more complex, especially where data moves rapidly from one function to another.





STRUCTURAL PARTITIONING: VERTICAL

In vertical partitioning, the functionality is distributed among the modules in a top-down manner. The modules at the top level called control modules perform the decision-making and do little processing whereas the modules at the low level called worker modules perform all input, computation and output tasks.





SUMMARY

- Modularisation is the process of dividing a program's functionality into independent modules.
- Cohesion describes the extent to which the functions in a module work together to achieve the ultimate aim of the module.
- Refactoring is the process of modifying a software system to improve the internal structure of design without changing its external behaviour.



FURTHER READING RESOURCES

- RUSS, F. (2002). EBOOK: Information Systems Development: Methods-in-Action. McGraw Hill.
- Riolo, R., Worzel, W. P., & Kotanchek, M. (Eds.). (2015). Genetic programming theory and practice XII. Springer International Publishing.
- DECOMPOSITION AND MODULARITY.
<https://www.tutorsglobe.com/homework-help/software-engineering/decomposition-and-modularity-7779.aspx>



DEPARTMENT OF
COMPUTER SCIENCE



THANK
YOU